

数据读取与合并

样本类型

- **Normal**: 正常组织
 - 来源于该器官的未发生癌变的正常细胞
 - 探究**基准**
- **Primary**: 原发灶, 原发性肿瘤
 - 肿瘤最初生长的部位所形成的肿瘤组织
 - 探究**发生**
- **Metastatic**: 转移灶, 转移性肿瘤
 - 肿瘤细胞脱离原发灶, 通过血液或淋巴系统移动到了身体的另一处器官 (如肝脏、骨骼、大脑、淋巴结) 并重新生长出来的肿瘤
 - 探究**恶化**

10x 格式的三个文件

- matrix.mtx.gz
 - 表达矩阵本体, 稀疏格式, 3 行头部 + 数据, 如下
 - 从第 4 行开始, 每行只记一个非零值

```
%%MatrixMarket matrix coordinate integer general
%metadata_json: {...}
33538 11786 23182236
33509 1 64
33507 1 8
...
```

← 格式声明: 坐标式整数稀疏矩阵
← 注释 (软件版本信息, 以 % 开头)
← 关键: 基因数 细胞数 非零元素数
← 第 33509 号基因, 第 1 号细胞, 计数 64
← 第 33507 号基因, 第 1 号细胞, 计数 8

- barcodes.tsv.gz
 - 仅一列, 格式为(n_obs, 1)
 - 每行一个细胞的标签, 如AAACCCAAGGATGTTA-1, 其中'-1'代表这是第一个样本的细胞
- feature.tsv.gz
 - 三列, 格式为(n_var, 3)
 - 分别为: Ensembl 基因 ID、基因符号、特征类型, 如

ENSG00000243485	MIR1302-2HG	Gene Expression
ENSG00000237613	FAM138A	Gene Expression

```

sample_dirs = {                                     # 字典:样本名 → 路径
    'primary':    r'd:\SingleCell\references\study_data\01_QC\matrix\primary',
    'metastatic': r'd:\SingleCell\references\study_data\01_QC\matrix\metastatic',
    'normal':     r'd:\SingleCell\references\study_data\01_QC\matrix\normal',
}

adatas = {}                                         # 空字典:准备装三个 AnnData
for name, path in sample_dirs.items():             # 遍历:name='primary', path=对应路径
    adata = sc.read_10x_mtx(path, var_names='gene_symbols', cache=True)
    # 将矩阵、barcodes和features组装成一个AnnData对象
    adata.obs['sample'] = name                      # 给这组数据的每个细胞打上样本标签
    adatas[name] = adata                           # 存进字典:adatas['primary'] = AnnData

adata = ad.concat(adatas, label='sample')          # 三合一:细胞数相加,基因列对齐
adata.obs_names_make_unique()                     # 细胞名去重(不同样本细胞名可能重复)
adata                                              # 输出摘要,看形状

```

质量控制 QC Quality Control

目的：识别并剔除质量差的细胞

类型	特征	产生原因
空微滴	检测到的基因极少，总 UMI 极低	微滴里没包到细胞，只测到背景 RNA
破损/死细胞	线粒体基因占比高	细胞膜破裂，mRNA 泄漏，只剩线粒体 RNA
Doublet	检测到的基因异常多，总 UMI 异常高	一个微滴包了两个细胞
红细胞污染	血红蛋白基因占比高	血液样本中红细胞未充分裂解

流程

1. 算指标

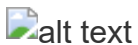
- 新增列以标记线粒体基因，True则表明标记
- 调用函数sc.pp.calculate_qc_metrics()来计算三个**核心QC指标**
 - n_genes_by_counts** 表达量 >0 的基因数；
 - total_counts** 每个细胞的总计数；
 - pct_counts_mt** 线粒体基因计数所占百分比。

```
# mitochondrial genes, "MT-" for human, "Mt-" for mouse
adata.var["mt"] = adata.var_names.str.startswith("MT-") # 标记线粒体基因，如果以MT-开头，就标为True

sc.pp.calculate_qc_metrics(
    adata,
    qc_vars=["mt"], # 对mt基因算占比
    inplace=True, # 直接写回adata.obs
    log1p=True # 生成对数化版本
)
```

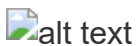
2. 看分布

```
sc.pl.violin(
    adata,
    ["n_genes_by_counts", # 多少个基因表达量 > 0
     "total_counts", # 细胞总 UMI 数，或者说是原始 RNA 分子数
     "pct_counts_mt"], # 线粒体基因占总 UMI 的百分比
    jitter=0.4,
    multi_panel=True,
)
```



此外，可以用 `pct_counts_mt` 着色的散点图联合考察这些 QC 指标。

```
sc.pl.scatter(adata, "total_counts", "n_genes_by_counts", color="pct_counts_mt")
```



颜色越靠近黄绿色，则线粒体基因占比越高，表明细胞破裂越严重，应当过滤掉。

3. 设置阈值并过滤

过滤分两步：

- **细胞：**
 - 滤掉基因数太少的细胞，如基因数不足100的细胞
 - 这些可能是空液滴 / 死细胞
 - 按行过滤
- **基因：**
 - 滤掉只在极少数细胞里出现的基因，如在不足3个细胞里检测到的

- 大概率是噪声基因，无生物学意义
- 按列过滤

由于单细胞数据的基因是 **长尾分布**，即少数基因高度表达，大量基因几乎沉默。所以往往对基因能过滤掉更多数量

```
sc.pp.filter_cells(adata, min_genes=100) # 过滤掉 0.6% 的细胞
sc.pp.filter_genes(adata, min_cells=3)   # 过滤掉 35% 的基因
```

一般不通过 `total_counts`，即总计数来筛选。因为该指标反映的是测序深度，不代表细胞质量

还应注意：对于含多个批次的数据集，应分别对每个样本进行质量控制，因为不同批次的 QC 阈值可能相差很大。

双细胞检测 Doublet detection

- **问题**：双细胞可能会导致错误分类
- **目标**：识别双细胞
- **方法一**
 - **模拟**：利用现有表达谱，随机合成一批假的doublet
 - **判别**：训练分类器，判断细胞更像真实单细胞还是模拟双细胞
 - **输出**两列到 `.obs`，可据此过滤
 - `doublet_score`
 - 双细胞概率分数
 - 0 ~ 100%
 - `predicted_doublet`
 - 判断是否为双细胞
 - True / False

```
sc.pp.scrublet(adata, batch_key="sample")
```

- **方法二**：先留着，聚类后看看哪些 cluster 的 `doublet_score` 高，再处理

归一化 Normalization

- **问题**：不同细胞总 UMI 不一致，不可直接比较，避免因 UMI 不一致（即测序深度不一致）带来的测序深度差异
- **目的**：解决细胞之间（即测序深度）带来的差异
- **常见做法**

i. 按比例缩放 `normalized_total`

◦ 将数据标准化到某个缩放因子，如

- 中位数
 - `pp.normalize_total`默认
 - $\frac{count_{gene}}{\text{中位数}}$
- 10000 (CP10k)
 - `target_sum=1e4`
 - $\frac{count_{gene} \times 10000}{total_counts}$
- 1000000 (CPM)
 - `target_sum=1e6`
 - $\frac{count_{gene} \times 1000000}{total_counts}$

ii. 对数变换 `log1p`

- 必要性：归一化后往往数值范围很大，且极度右偏
 - 少数基因表达极高
 - 大量基因接近于 0
- 对每个值做 $\ln(1 + x)$ ，缩小基因间差距

用counts这个layer存放计数矩阵

```
adata.layers["counts"] = adata.X.copy()
```

对adata标准化到中位数

```
sc.pp.normalize_total(adata)
```

对数转换

```
sc.pp.log1p(adata)
```

特征选择

- **问题**：大多数基因在所有细胞中表达量差不多，难以区分细胞类型，需要被剔除
- **目标**：降低数据集维度，只保留信息量最大的基因，称作 **高变基因 HVG**
- **代码**：`pp.highly_variable_genes` 可根据所选flavor对高变基因进行标注。通常选2000

筛选前2000个高变基因，并处理批次效应，通常是分组计算、独立筛选，最后取并集

```
sc.pp.highly_variable_genes(adata, n_top_genes=2000, batch_key="sample")
```

画图

```
sc.pl.highly_variable_genes(adata)
```

- 坐标轴
 - 横轴：基因平均表达量
 - 纵轴：基因离散度
- 点颜色
 - 黑点：HVG 高变基因
 - 灰点：其他基因
- 子图区别
 - 左图：归一化处理，优先使用
 - 右图：未归一化处理

PCA降维

- PC是基因的加权组合，如 $PC1 = 0.8 \times CD3D - 0.4 \times MS4A1 + 0.2 \times NKG7 - \dots$
 - PC1 是数据里方差最大的方向
 - PC2 是方差第二大的方向，与PC1正交
- PCA可以揭示主要变异轴，并对数据进行去噪，**压缩冗余**（例如将一起涨的基因合并成一个PC，但信息没丢）
- 观察各主成分对数据总方差的贡献，有利于判断在计算细胞领域关系时应该考虑多少个PC。
 - 可以适当增加PC数
 - 这些领域关系可用于聚类
- 可以绘制主成分，检查是否可能有不希望出现的特征驱动了显著变异

```
# 计算PCA
sc.tl.pca(adata)

# 绘制方差比率/碎石图，log=True是指y轴取对数
sc.pl.pca_variance_ratio(adata, n_pcs=50, log=True)

# 绘制PCA降维散点图，检查技术变量
sc.pl.pca(
    adata,
    # 第 1~4 个面板的颜色
    color=["sample", "sample", "pct_counts_mt", "pct_counts_mt"],
    # 第 1~4 个面板的PC组合
    dimensions=[(0, 1), (2, 3), (0, 1), (2, 3)],
    ncols=2,
    size=2,
)
```

碎石图

- 横轴：第几个PC
- 纵轴：该PC的方差占比
- Elbow Point
 - 曲线开始变平缓的地方
 - 之后的PC通常是噪声

PCA降维散点图

面板	坐标	着色	检查的问题
左上	PC1 vs PC2	sample	最大变异轴是否按样本分开（批次效应）？
右上	PC3 vs PC4	sample	次大变异轴是否按样本分开？
左下	PC1 vs PC2	pct_counts_mt	最大变异轴是否被线粒体比例（QC）驱动？
右下	PC3 vs PC4	pct_counts_mt	次大变异轴是否被 QC 驱动？

结果

- PC1 与 PC2 可以较好地区分 normal、primary 和 metastatic 三类样本
- 样本间最大的变化主要集中在 PC1、PC2，而 PC3、PC4 解释的是次级变化，且变化较为不明显

neighborhood graph 的构建与可视化

目的：作为聚类的输入

使用数据矩阵的 PCA 表示来计算细胞的邻域图，因为PCA的结果经过去噪和降维，构建更快更准

步骤

1. kNN 找邻居
 2. 连边 + 加权，越相似，权重越高
- 得到邻域图，是数据内部的结构
 - 可以指定每个细胞连多少个邻居细胞，以及用前多少个PC来算距离，而非2000个HVG

```
sc.pp.neighbors(adata)
```

3. 用UMAP将邻域图嵌入二维空间

```
sc.tl.umap(adata)
```

4. 按sample可视化UMAP

```
sc.pl.umap(  
    adata,  
    color = "sample",  
    # 设置点的大小  
    size = 2,  
)
```

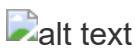


聚类——Leiden图聚类

- 该聚类直接对**细胞邻域图**进行聚类
- 可以选择分辨率 resolution。resolution越小，则cluster数越小，一个cluster的细胞数越大
- 优化目标：**模块度**
 - 簇内连接越密
 - 簇间连接越疏

```
sc.tl.leiden(  
    adata,                # AnnData 对象  
    flavor="igraph",      # 使用 igraph 库的实现（比默认更快）  
    n_iterations=2        # 迭代次数设为 2（加快运行速度）  
)
```

```
sc.pl.umap(adata, color=["leiden"]) # 画图
```



重新评估 QC 与细胞过滤

由上图可知，结果很不理想，需要从 **双细胞** 和 **线粒体比例** 入手，进行第二轮过滤

- 某簇**双细胞**得分高 → 可能是双细胞群，需剔除
 - leiden：参考图
 - predicted_doublet：双细胞图
 - 全为False，说明无双细胞

- doublet_score: 双细胞得分图
 - 大部分是紫色的，少部分偏黄绿

```
sc.pl.umap(
    adata,
    color=["leiden", "predicted_doublet", "doublet_score"],
    # increase horizontal space between panels
    wspace=0.5,
    size=3,
)
```



为什么在聚类前双细胞检测，但却在聚类后删除双细胞？

答：doublet_score是连续分数，高分细胞里可能有真单细胞。聚类之后，由于双细胞是两个细胞的混合体，所以双细胞之间彼此更像，往往会聚类到一个Cluster。此时得到的doublet_score不再是指针对一个细胞，而是一个Cluster。

- 某簇**线粒体比例**高 / 基因数低 → 可能是受损细胞，QC 没滤干净
 - leiden: 参考图
 - log1p_total_counts: UMI数量
 - 每个细胞测到了多少 RNA?
 - pct_counts_mt: 线粒体比例
 - log1p_n_genes_by_counts: 检测到的基因数

```
sc.pl.umap(
    adata,
    color=["leiden", "log1p_total_counts", "pct_counts_mt", "log1p_n_genes_by_counts"],
    wspace=0.5,
    ncol=2,
)
```



手动细胞注释

聚类给了一堆Cluster编号，但是编号无意义，需要对编号进行注释，其本质依据为 **marker gene**——某些标记基因只被特定细胞类型表达，也因此要先使用leiden来从 neighborhood graph 识别细胞群开始：

```

for res in [0.02, 0.5, 2.0]:
    sc.tl.leiden(adata, key_added=f"leiden_res_{res:4.2f}", resolution=res, flavor="igraph")

sc.pl.umap(
    adata,
    color=["leiden_res_0.02", "leiden_res_0.50", "leiden_res_2.00"],
    legend_loc="on data",
)

```

三要点

- 要点 1: 聚类的是"图", 不是原始数据, 也不是 UMAP 坐标
 - Leiden 直接在邻域图上工作, 图是唯一输入。UMAP 只是"看图用的投影", 不能拿 UMAP 坐标当聚类依据。
- 要点 2: 分辨率 resolution 参数控制聚类颗粒度
 - resolution 小 (如 0.02) → 簇少而大 (粗粒度, 可能把不同身份归到一起)
 - resolution 大 (如 2.0) → 簇多而碎 (细粒度, 可能把一类切碎)
 - 教程用 for res in [0.02, 0.5, 2.0] 跑三个分辨率并排对比, 选一个合适的 (教程里 0.50 合适)。
- 要点 3: 聚类数没有"正确答案"
 - 聚类数在很大程度上是任意的, resolution 参数也是如此。
 - 最终应由"能够稳定区分、且具有生物学意义的群体"来约束——这需要领域知识/标记基因先验, 不是算法自动给出的。



以下取分辨率为0.02为例来分析, 一共两种找 marker 的方法

方法A: Marker Gene 集

1. 用整理好的 "细胞类型": ["Marker_Gene_1", "Marker_Gene_2", ...] 清单, 检查每个Cluster表达哪些 marker
2. 在粗聚类上画 dotplot, 看谱系

```

marker_genes= {
    "CD4+ T": ["TRAC", "CCR7"],
    "CD8+ T": ["CD8B", "CD8A", "GZMK"],
    "NK": ["NKG7", "KLRF1", "KLRD1", "CD247", "GZMK"],
    "NKT": ["NKG7", "GNLY", "KLRD1"],
    "B cell": ["MS4A1", "BANK1"],
    "Plasma cells": ["MZB1", "JCHAIN", "IGHG1"],
    "Myeloid": ["LYZ", "CD14", "S100A8", "S100A9", "FCN1", "CD68", "C1QA", "C1QB", "MRC1", "SPP:
    "Fibroblast": ["COL1A1", "TAGLN", "ACTA2", "SPARC"],
    "Endothelial": ["CLDN5", "FLT1", "ENG", "PECAM1", "AQP1", "EGFL7"],
    "Epithelial": ["EPCAM", "KRT8", "KRT18", "KRT19", "CD24", "ELF3", "MUC1"],
    "Proliferation": ["MKI67", "TOP2A"],
}

sc.pl.dotplot(
    adata,
    marker_genes,
    groupby="leiden_res_0.02", # 得到4个Cluster
    standard_scale="var" # 按基因进行Z-score标准化
)

```

 alt text

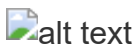
- 圆圈的大小 (Fraction of cells in group %)
 - 代表该群中有多少比例的细胞表达了该基因。
 - 圈越大 (如 80% - 100%)，说明这个基因在这个群里是普遍表达的。
- 圆圈的颜色 (Mean expression in group):
 - 代表该群细胞中该基因的平均表达水平。
 - 颜色越红 (接近 1.0)，表达量越高；
 - 颜色越白 (接近 0.0)，表达量越低。
- 最上面的小标题
 - 为了方便理解，代码将基因按“**细胞类型**”归了类
 - 比如 CD4+ T 下方就是 T细胞的标志基因)

3. 映射——建字典，贴标签

```
# 1. 建立字典查表
mapping = {
    "0": "Myeloid",
    "1": "CD8 T cell",
    "2": "Fibroblast",
    "3": "Epithelial"
}

# 2. 精确替换, 产出新列 cell_type_lv1(即level 1, 第一层注释), 说明每个细胞属于哪个谱系
adata.obs["cell_type_lv1"] = adata.obs["leiden_res_0.02"].map(mapping)

# 画图, 但是分辨率提高, 做第二层注释
sc.pl.dotplot(adata, marker_genes, groupby="leiden_res_0.50", standard_scale="var")
```



特点: 先在 0.02 (粗) 上贴 cell_type_lv1, 再在 0.50 (细) 上画 dotplot

原因:

- 粗聚类回答"这是什么谱系" (稳, 靠比例)
- 细聚类回答"谱系里具体是什么细胞" (精, 靠标记基因)

方法B: 用差异表达基因DE genes作Marker Gene

此外, 可以计算每个Cluster的Marker Gene, 检查这些Gene能否与已知生物学关联 (如细胞类型或状态)。

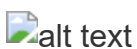
1. 算DE基因:

- 选择分辨率
- 通常对每个Cluster进行 Wilcoxon检验 或 t检验, 即让每个 Cluster 对其余所有 Cluster 作比较

2. 画 dotplot 看 top 5 基因

```
sc.tl.rank_genes_groups(adata, groupby="leiden_res_0.02", method="wilcoxon")
# 使用Wilcoxon检验, 对 每个Cluster vs 其他Cluster 做差异表达, 结果存进adata.uns["rank_genes_groups"]

sc.pl.rank_genes_groups_dotplot(adata, groupby="leiden_res_0.02", standard_scale="var", n_genes=
```



3. 用基因反推细胞类型

- 如 Cluster 3 的top 5 DE Genes为INS, IGKC, IGLC2, PRSS2, IGHA1，可以判断为 内分泌/浆细胞混合

4. 提取 DE 基因为DataFrame，便于处理

5. 把 top 基因画在 UMAP 上验证

```
sc.get.rank_genes_groups_df(adata, group="3").head(5)
```

```
dc_cluster_genes = sc.get.rank_genes_groups_df(adata, group="3").head(5)["names"]
```

```
sc.pl.umap(
    adata,
    color=[*dc_cluster_genes, "leiden_res_0.02"],
    legend_loc="on data",
    frameon=False,
    ncols=3,
)
```

names	scores	logfoldchanges	pvals	pvals_adj
INS	121.203270	13.284445	0.0	0.0
IGKC	120.921043	7.425559	0.0	0.0
IGLC2	119.367393	7.332287	0.0	0.0
PRSS2	119.047386	9.433443	0.0	0.0
IGHA1	118.821342	7.683056	0.0	0.0

- names
 - 基因名
 - 按照差异显著程度从高往低排
- scores
 - 检验**统计量**，反映**差异是否显著**
 - Wilcoxon 检验算出的统计量 U 值
 - 得到这个Cluster vs 其他Clusters，表达差异有多大
- logfoldchanges / Log2FC
 - log2倍数变化，反映**差异的大小**
 - $\log_2\left(\frac{\text{DeGene在本Cluster的平均表达}}{\text{DeGene在其他Cluster的平均表达}}\right)$
- pvals
 - P值，反映**差异的可信度**
 - H_0 : 本Cluster表达 = 其他Cluster
 - H_1 : 本Cluster表达 $\neq$ 其他Cluster

- H_0 成立的概率，越小则差异越可信
- 但是，p值往往极低，因为把每个细胞当初独立样本后，样本量很大，不代表差异巨大，需要看 `pvals_adj`
- `pvals_adj`
 - 矫正后的P值，防假阳性
 - 可以使用多重检验校正等方法，调高p值，控制假阳性率

 alt text

由图可见，Cluster 3的top 5 DE Genes在其他Cluster里几乎不表达